

Kaskadowe arkusze stylów CSS

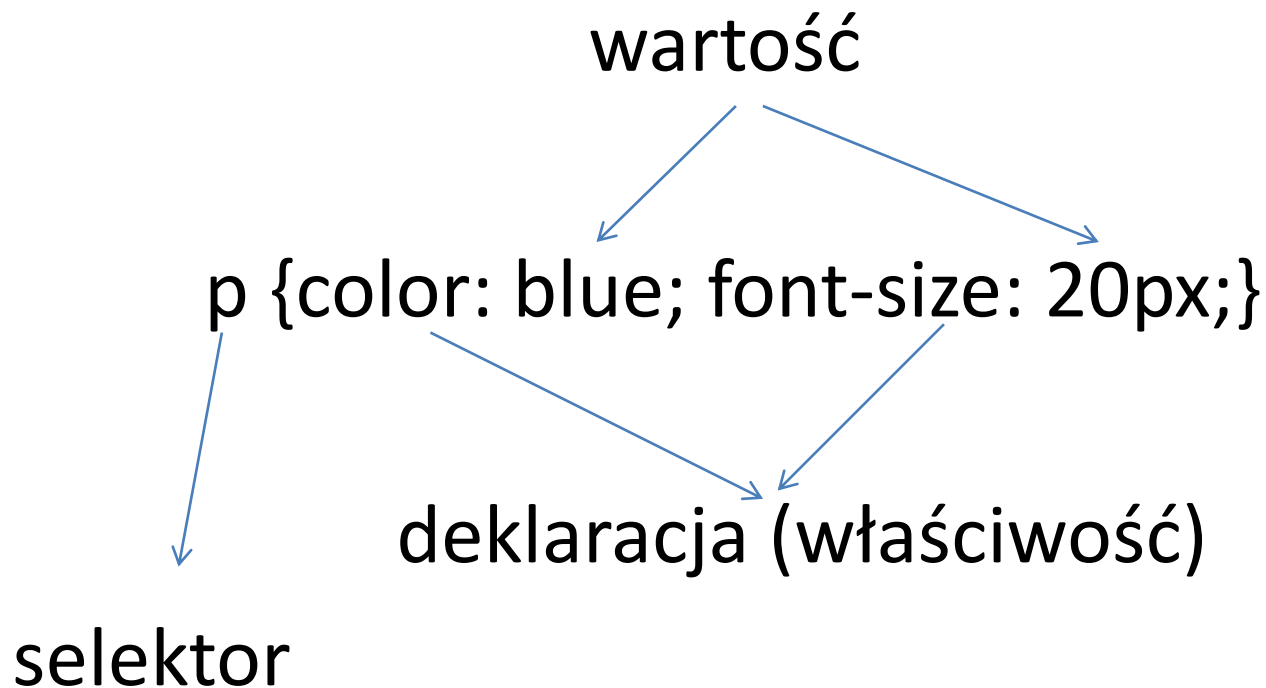
CSS - wprowadzenie

- CSS (Cascading Style Sheets) – Kaskadowe Arkusze Stylów
- Język CSS powstał do określania układu graficznego dokumentu HTML. Wszystkie polecenia dotyczące formatowania wyglądu powinny zostać umieszczone w osobnym pliku i powiązane z elementami zdefiniowanymi w kodzie HTML.

Cechy języka CSS

- Oddziela strukturę informacyjną witryny od jej warstwy prezentacyjnej
- Daje większe możliwości formatowania tekstu
- Pozwala zapisać wszystkie informacje dotyczące wyglądu dokumentu w oddzielnym pliku tekstowym dołączonym później do dokumentów HTML
- Umożliwia formatowanie wielu dokumentów przy pomocy jednego arkusza CSS
- Umożliwia stosowanie różnych układów graficznych w zależności od typu urządzenia na którym zamierzamy wyświetlić projektowaną stronę

Podstawowa składnia kaskadowego arkusza stylów



Podstawowa składnia kaskadowego arkusza stylów

- **Selektor** – określa do jakich znaczników języka HTML odnosi się dalsza część definicji stylu. Selektorem może być dowolny znacznik HTML np. p, body, li, h1. Selektor służy do wskazania dla jakiego elementu chcemy ustawić styl.
- **Właściwość** – służy do określenia jaki atrybut znacznika będziemy ustawiali np. wielkość czcionki, kolor czcionki, rodzaj czcionki
- **Wartość** – określa konkretną wartość atrybutu np. 20px, blue, #348282

Reguły obowiązujące w składni kaskadowych arkuszy stylów

- Selektor wskazuje na element w których chcemy zmienić styl
- Blok deklaracji zawiera jedną lub kilka deklaracji rozdzielonych średnikami
- Każda deklaracja zawiera nazwę właściwości oraz po dwukropku wartość przypisaną tej właściwości
- Każda deklaracja kończy się średnikiem
- Blok deklaracji jest otoczony nawiasami klamrowymi

Sposoby dołączania stylów

- Styl lokalny (w linii)
- Styl wewnętrzny
- Styl zewnętrzny (z pliku)

Styl lokalny

- Korzystając ze stylu lokalnego, można zdefiniować formatowanie pojedynczego elementu strony. Taki styl definiowany jest w tej samej linii, w której znajduje się element formatowany, dlatego nazywany jest on również stylem w linii. Umieszczany jest on bezpośrednio w dokumencie HTML. Wadą tego sposobu jest mieszanie warstwy struktury i treści z warstwą prezentacji. Powinien być on stosowany tylko w wyjątkowych sytuacjach.

```
<body style="background-color: #df1092;">
```

```
<p style="color: blue; font-size: 12px"> nasz tekst </p>
```

Znacznik span

Jeżeli chcemy wyróżnić fragment tekstu używamy znacznika span.

```
<p> TEKST NORMALNY <span style="color:
blue; font-size: 12px"> TEKST WYRÓŻNIONY
</span> TEKST NORMALNY </p>
```

Znacznik div

Znacznika div używamy aby pogrupować elementy na stronie internetowej. Zdefiniowane bloki mogą zawierać akapity, listy, tabele oraz inne bloki. Znacznik div umożliwia wydzielenie na stronie większych, logicznych fragmentów w celu nadania im specyficznego formatowania z użyciem stylów.

```
<div style="właściwość: wartość;"> ... </div>
```

Np.

```
<div style="color: blue; font-size: 12px">  
<h1>Tytuł</h1>  
<p>Pierwszy akapit</p>  
<p>Drugi akapit</p>  
</div>
```

Wewnętrzny arkusz stylów

Wewnętrzny arkusz stylów jest umieszczany w dokumencie HTML dzięki zastosowaniu znacznika `<style>`, który zawsze występuje w części nagłówkowej dokumentu (`<head>`). Tę metodę wstawiania do dokumentu arkusza stylów stosuje się, gdy elementy formatowane pojawiają się na stronie wielokrotnie i wszystkie powinny mieć takie same atrybuty formatowania.

Wewnętrzny arkusz stylów

```
<!doctype HTML>
<HTML lang="pl-PL">
<HEAD>
<style>
p{
    color:#237722;
    text-align:center;
    font-size:70px;
}
</style>
<META charset="UTF-8">
<TITLE>CSS</TITLE>
</HEAD>
<BODY>
<p>Tekst testowy</p>
<p>Nowy tekst</p>
<p>Kolejny tekst</p>
</BODY>
</HTML>
```

Zewnętrzny arkusz stylów

Dodanie zewnętrznego arkusza stylów polega na utworzeniu pliku tekstowego z rozszerzeniem .css, który będzie zawierał definicję wszystkich stylów używanych w projektowanej witrynie. W dokumencie HTML powinien znaleźć się odnośnik do tzw. zewnętrznego arkusza stylów, czyli do pliku z rozszerzeniem .css.

Odnośnik powinien zostać umieszczony w sekcji <head> i mieć postać:

```
<link rel="stylesheet" type="text/css" href="nazwa.css">
```

Zewnętrzny arkusz stylów – przykład dołączenia stylu

```
1  <!DOCTYPE html>
2  <html lang="pl-PL">
3  <head>
4    <link rel="stylesheet" type="text/css" href="moj.css">
5  </head>
6  <body>
7    <h1>nowy</h1>
8
9    <h1>MOJA NOWA STRONA</h1>
10   <p>TEKST PIERWSZY</p>
11   <p>TEKST DRUGI</p>
12
13 </body>
14 </html>
15
```

Zewnętrzny arkusz stylów – przykład pliku css

```
1  p{
2      color:#237722;
3      text-align:center;
4      font-size:70px;
5  }
6  body{
7      background-color:rgb(100,200,100);
8  }
9  /*
10     komentarz
11  */
12  h1{
13      color:rgba(10,170,20,0.9);
14      text-align:center;
15      font-size:70px;
16  }
17  }
```

Import arkusza stylów

Do zewnętrznego lub wewnętrznego arkusza stylów można zaimportować zewnętrzny arkusz stylów. Oprócz importowania arkusza można między znacznikami `<style>` i `</style>` umieszczać inne deklaracje stylów. Polecenie importowania arkusza muszą występować na początku arkusza stylów, przed właściwymi regułami CSS.

```
<style>
```

```
@import url("nazwa.css");
```

```
</style>
```

Kaskadowość arkuszy stylów

- Styl lokalny
- Wydzielone bloki
- Wewnętrzny arkusz stylów
- Zewnętrzny arkusz stylów
- Import stylów do zewnętrznego arkusza

Zmiana kaskadowości stylów

Aby zmienić kaskadowość stylów możemy użyć polecenia `!important` umieszczonego w deklaracji stylu po wartości, której dotyczy. Takie umiejscowienie tego polecenia spowoduje, że właściwość zdefiniowana dla elementu będzie miała pierwszeństwo, dla której zostało użyte, pozostałe właściwości elementu będą obsługiwane zgodnie z ogólnymi zasadami kaskadowości stylów.

```
<p style="color: blue !important; font-size: 12px">  
nasz tekst </p>
```

Kaskadowość - przykład

```
1  <!doctype HTML> <!--4.html-->
2  <HTML lang="pl-PL">
3  <HEAD>
4  <style>
5      .nowaklasa{
6          color:#237722;
7          text-align:center;
8          font-size:70px;
9      }
10 </style>
11 <META charset="UTF-8">
12 <TITLE>CSS</TITLE>
13 </HEAD>
14 <BODY>
15 <div class="nowaklasa">
16     <p>Nowy tekst</p>
17     <p style="color:red;">Kolejny tekst</p>
18     <p>Następny tekst</p>
19     <h1>Inny tekst</h1>
20 </div>
21 </BODY>
22 </HTML>
```

Zmiana kaskadowości - przykład

```
1  <!doctype HTML> <!--4a.html-->
2  <HTML lang="pl-PL">
3  <HEAD>
4  <style>
5  .nowaklasa p{
6      color:#237722 !important;
7      text-align:center !important;
8      font-size:70px;
9  }
10 </style>
11 <META charset="UTF-8">
12 <TITLE>CSS</TITLE>
13 </HEAD>
14 <BODY>
15 <div class="nowaklasa">
16 <p>Nowy tekst</p>
17 <p style="color:red; text-align:left;">Kolejny tekst</p>
18 <p>Następny tekst</p>
19 <h1>Inny tekst</h1>
20 </div>
21 </BODY>
22 </HTML>
```

Dziedziczenie w CSS

W języku HTML często występują zagnieżdżenia jednego elementu wewnątrz drugiego. Jeżeli dla elementu nadrzędnego w arkuszach stylów zostały zdefiniowane właściwości, to w większości przypadków elementowi podrzędnemu zostaną przypisane te same właściwości, nawet jeżeli nie zostały dla niego zdefiniowane. Mechanizm ten nazywamy dziedziczeniem.

Dziedziczenie – przykład

```
1  <!doctype HTML> <!--5.html-->
2  <HTML lang="pl-PL">
3  <HEAD>
4  <style>
5    body {
6      color:#237722;
7      text-align:center;
8      font-size:70px;
9    }
10 </style>
11 <META charset="UTF-8">
12 <TITLE>CSS</TITLE>
13 </HEAD>
14 <BODY>
15 <p>Teskt testowy</p>
16 <div>
17 <p>Nowy <span>tekst</span></p>
18 <p>Kolejny tekst</p>
19 <h1>Wpis</h1>
20 </div>
21 <p>Teskt testowy</p>
22 </BODY>
23 </HTML>
```

Wartości stosowane w stylach CSS

- Liczby – całkowite lub rzeczywiste. Liczba całkowita składa się z cyfr od 0 do 9. Liczba rzeczywista może zawierać część całkowitą i ułamkową zapisaną po kropce.
- Procenty – zawsze są określane względem innej wartości (np. wielkości okna przeglądarki)
- Słowa kluczowe – nie zawsze wartości reprezentowane przez słowa kluczowe są poprawnie interpretowane przez przeglądarki (np. blue, red)

Wartości stosowane w stylach CSS

- Wartości bezwzględne:
 - **in** (cal, 1 in = 2,54 cm)
 - **cm** (centymetr)
 - **mm** (milimetr)
 - **pt** (punkt, 1 cal to 72 punkty)
 - **pc** (pica, 1pc = 12 pt)
 - **px** (piksel, 1 px = 0,75 pt)
- Wartości względne
 - **em** (1 em jest wielkością rozmiaru czcionki, jaką posiada dany element. Wartość 0.1 em jest wartością najmniejszą jaką przeglądarka może rozpoznać)
 - **ex** (obliczana na podstawie wysokości małej litery x użytej w czcionce)
- Jednostki czasu to: **s** lub **ms**

Wartości stosowane w stylach CSS

```
1  <!DOCTYPE html> <!--7.html-->
2  <html lang="pl-PL">
3  <head>
4    <meta charset="UTF-8"/>
5  </head>
6  <body>
7    <p>Wartości bezwzględne.</p>
8    <p style="font-size: 30px;">Tekst testowy 30px</p>
9    <p style="font-size: 30pt;">Tekst testowy 30pt</p>
10   <p style="font-size: 3pc;">Tekst testowy 3pc</p>
11   <p style="font-size: 0.5in;">Tekst testowy 0.5in</p>
12   <p style="font-size: 1cm;">Tekst testowy 1cm</p>
13   <p style="font-size: 10mm;">Tekst testowy 10mm</p>
14   <p>Wartości względne. (zalecane)</p>
15   <p style="font-size: 5em;">Tekst testowy 5em</p>
16   <p style="font-size: 5ex;">Tekst testowy 5ex</p>
17 </body>
18 </html>
19
```

Wartości stosowane w stylach CSS

Wartości bezwzględne.

Teskt testowy 30px

Teskt testowy 30pt

Teskt testowy 3pc

Teskt testowy 0.5in

Teskt testowy 1cm

Teskt testowy 10mm

Wartości względne. (zalecane)

Teskt testowy 5em

Teskt testowy 5ex

Wartości kolorów w CSS

Kolor	Nazwa	Kod HEX	Kod RGB
czarny	black	#000000	rgb(0,0,0)
srebrny	silver	#c0c0c0	rgb(192,192,192)
szary	gray	#808080	rgb(128,128,128)
biały	white	#ffffff	rgb(255,255,255)
kasztanowy	maroon	#800000	rgb(128,0,0)
czerwony	red	#ff0000	rgb(255,0,0)
purpurowy	purple	#800080	rgb(128,0,128)
fukcja	fuchsia	#ff00ff	rgb(255,0,255)
zielony	green	#008000	rgb(0,128,0)
limonkowy	lime	#00ff00	rgb(0,255,0)
oliwkowy	olive	#808000	rgb(128,128,0)
żółty	yellow	#ffff00	rgb(255,255,0)
granatowy	navy	#000080	rgb(0,0,128)
niebieski	blue	#0000ff	rgb(0,0,255)
morski	teal	#008080	rgb(0,128,128)
akwamaryna	aqua	#00ffff	rgb(0,255,255)

Sposoby podawania kolorów w CSS

- **Predefiniowane** – podawanie kolorów w postaci nazw np. `p {color: orange;}`
- **#rrggbb** – podanie wartości każdej składowej kolorów podstawowych w systemie szesnastkowym. Do zapisu każdego z kolorów używamy po dwa znaki (rr – czerwony, gg – zielony, bb – niebieski) np. `p {color: #f0e982;}`
- **#rgb** – jako składowe podajemy liczby jednocyfrowe w systemie szesnastkowym
np. `p {color: 3f9;}`
- **rgb(R,G,B)** – można podać każdą składową koloru w systemie dziesiętnym, zakres wartości od 0 do 255
np. `p {color: rgb(255,23,33);}`

Sposoby podawania kolorów w CSS

- `rgb(R%, G%, B%)` – można podać określoną barwę w postaci procentowej
np. `p {color: rgb(10%,33%,100%);}`
- `hsl (H,S%,L%)` – H – brawa, S – nasycenie, L – jasność
np. `p {color: hsl(30,23%,33%);}`
- `rgba(R,G,B,A)`, `rgba(R%,G%,B%,A%)`, `hsla(H,S%,L%,A)` – A jest to parametr który umożliwia uzyskanie przezroczystości. Zakres wartości parametru A od 0 do 1.
- 0 – całkowita przezroczystość,
- 1 – brak przezroczystości
np. `p {color: rgba(30,23,33,0.5);}`
np. `p {color: hsla(30,23%,33%,0.5);}`

Selektor uniwersalny

- Selektor uniwersalny to selektor pasujący do wszystkich znaczników. Jest oznaczany *
- Składnia ogólna:
`*{właściwość: wartość;}`
- Zamiast stosować:
`p, h1, h2, h3, h4, h5, h6, table {color: blue;}`
można użyć selektora uniwersalnego:
`*{color: blue;}`

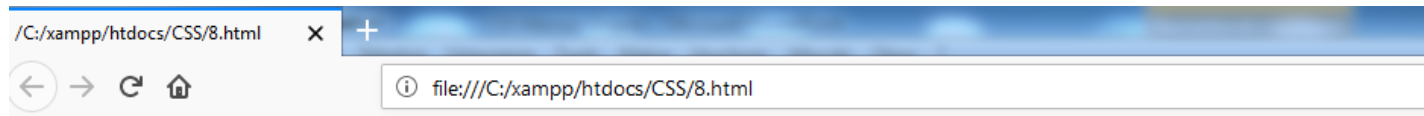
Selektor potomka

- Przy użyciu selektora potomka można formatować elementy, które są zawarte wewnątrz innych znaczników, czyli leżą niżej w hierarchii drzewa dokumentu.
- Składnia:
selektor selektor {właściwość: wartość;}
np. `div span {font-size: 20px; color: red;}`

Selektor potomka - przykład

```
1  <!DOCTYPE html> <!--8.html-->
2  <html lang="pl-PL">
3  <head>
4    <meta charset="UTF-8"/>
5    <style>
6      div span{
7        font-size: 20px;
8        color: red;
9      }
10   </style>
11   </head>
12   <body>
13
14   <div>
15     TEKST<span> SPAN TEKST</span>
16     <p>TEKST_P<span> SPAN TEKST_P</span></p>
17   </div>
18
19   <p>TEKST_BEZ_DIV<span> SPAN TEKST_BEZ_DIV</span></p>
20 </body>
21 </html>
```

Selektor potomka - przykład



TEKST SPAN TEKST

TEKST_P SPAN TEKST_P

TEKST_BEZ_DIV SPAN TEKST_BEZ_DIV

Selektor potomka - przykład

- Formatowane są tylko elementy umieszczone w znaczniku `<span>` który jest wewnątrz znacznika `<div>` lub w innych znacznikach znajdujących się w znaczniku `<div>` w tym przypadku w znaczniku `<p>`
- Jak widzimy tekst umieszczony w znaczniku `<span>` umieszczony poza znacznikiem `<div>` nie jest formatowany

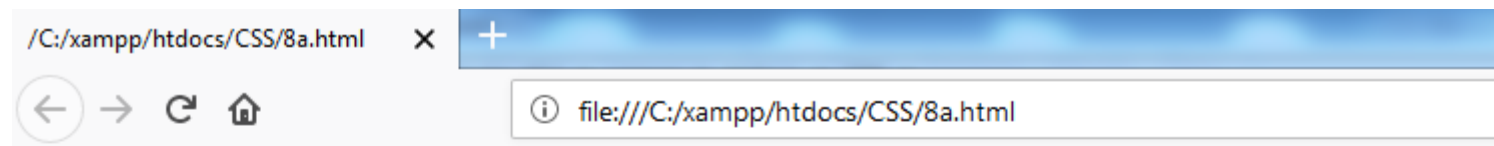
Selektor dziecka

- Selektor dziecka służy do definiowania formatowania elementów, które znajdują się o jeden rząd niżej w hierarchii drzewa dokumentu.
- Składnia:
rodzic > dziecko {właściwość: wartość;}

Selektor dziecka - przykład

```
1  <!DOCTYPE html> <!--8a|.html-->
2  <html lang="pl-PL">
3  <head>
4    <meta charset="UTF-8"/>
5    <style>
6      div > span{
7        font-size: 20px;
8        color: red;
9      }
10   </style>
11   </head>
12   <body>
13
14   <div>
15     TEKST<span> SPAN TEKST</span>
16     <p>TEKST_P<span> SPAN TEKST_P</span></p>
17   </div>
18
19   <p>TEKST_BEZ_DIV<span> SPAN TEKST_BEZ_DIV</span></p>
20 </body>
21 </html>
```

Selektor dziecka - przykład



TEKST SPAN TEKST

TEKST_P SPAN TEKST_P

TEKST_BEZ_DIV SPAN TEKST_BEZ_DIV

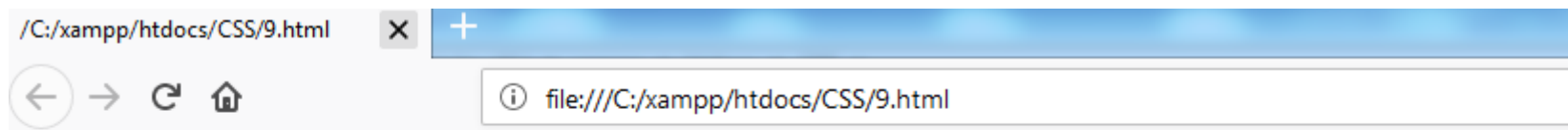
Selektor sąsiadującego brata

- Dla elementów znajdujących się w tym samym rzędzie hierarchii można zdefiniować selektor sąsiadującego brata. Umożliwia on nadanie drugiemu z sąsiadujących elementów zdefiniowanych atrybutów formatowania.
- Składania:
brat1 + brat2 {właściwość: wartość;}

Selektor sąsiadującego brata - przykład

```
1 <!DOCTYPE html> <!--8a.html-->
2 <html lang="pl-PL">
3 <head>
4 <meta charset="UTF-8"/>
5 <style>
6   div + p{
7     font-size: 20px;
8     color: red;
9   }
10 </style>
11 </head>
12 <body>
13
14 <div>
15   Litwo, Ojczyzno moja! ty jesteś jak <span> zdrowie</span>
16   <p>Ile cię trzeba cenić,<span> ten tylko się dowie,</span></p>
17 </div>
18
19 <p>Kto cię stracił. <span> Dziś piękność twą w całej ozdobie</span></p>
20 <p>Widzę i opisuję, bo tęsknię po tobie.</p>
21 </body>
```

Selektor sąsiadującego brata - przykład



Litwo, Ojczyzno moja! ty jesteś jak zdrowie

Ile cię trzeba cenić, ten tylko się dowie,

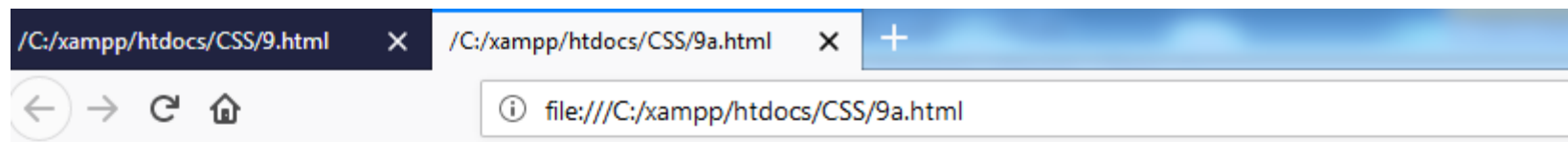
Kto cię stracił. Dziś piękność twą w całej ozdobie

Widzę i opisuję, bo tęsknię po tobie.

Selektor braci

- Dla elementów znajdujących się w tym samym rzędzie hierarchii można zdefiniować selektor braci. Umożliwia on nadanie drugiemu i wszystkim następnym sąsiadującym elementom zdefiniowanych atrybutów formatowania.
- Składnia `brat1 ~ brat2 {właściwość: wartość;}`

Selektor braci - przykład



Litwo, Ojczyzno moja! ty jesteś jak zdrowie

Ile cię trzeba cenić, ten tylko się dowie,

Kto cię stracił. Dziś piękność twą w całej ozdobie

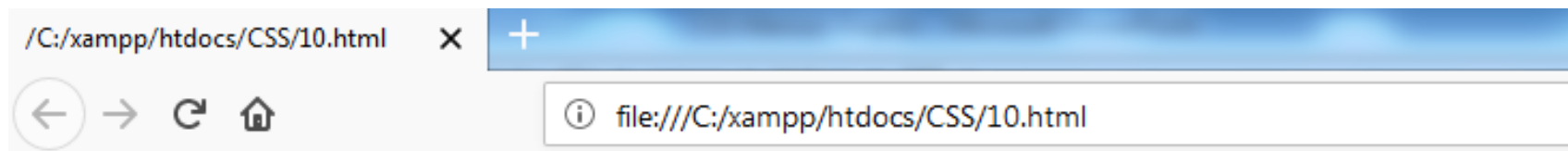
Widzę i opisuję, bo tęsknię po tobie.

Selektory atrybutów

- W języku CSS można formatować znaczniki na podstawie atrybutów, które one mają.
- Składnia:
 - Prosty selektor atrybutu:
`selektor[attribut] {właściwość: wartość;}`
 - Selektor atrybutu o określonej wartości
`selektor[attribut="wartość"] {właściwość: wartość;}`
 - Selektor atrybutu zawierający określony wyraz
`selektor[attribut~="wyraz"] {właściwość: wartość;}`

```
1 <!DOCTYPE html> <!--10.html-->
2 <html lang="pl-PL">
3 <head>
4   <meta charset="UTF-8"/>
5   <style>
6     p[id]{
7       font-size: 20px;
8       color: green;
9     }
10    h1[id="lewy"]{
11      font-size: 20px;
12      color: green;
13    }
14    h2[id~="cos"]{
15      font-size: 20px;
16      color: green;
17    }
18  </style>
19 </head>
20 <body>
21
22
23   <p id> Litwo, Ojczyzno moja! ty jesteś jak zdrowie</p>
24   <p>Ile cię trzeba cenić, ten tylko się dowie,</p>
25
26
27   <H1 id="lewy">Kto cię stracił. Dziś piękność twą w całej ozdobie</h1>
28   <H1 id="prawy">Kto cię stracił. Dziś piękność twą w całej ozdobie</h1>
29   <h2 id="Tutaj cos jest">Widzę i opisuję, bo tęsknię po tobie.</h2>
30   <h2 id="A tu nie ma">Widzę i opisuję, bo tęsknię po tobie.</h2>
31 </body>
32 </html>
```

Selektor atrybutów – efekt działania



Litwo, Ojczyzno moja! ty jesteś jak zdrowie

Ile cię trzeba cenić, ten tylko się dowie,

Kto cię stracił. Dziś piękność twą w całej ozdobie

Kto cię stracił. Dziś piękność twą w całej ozdobie

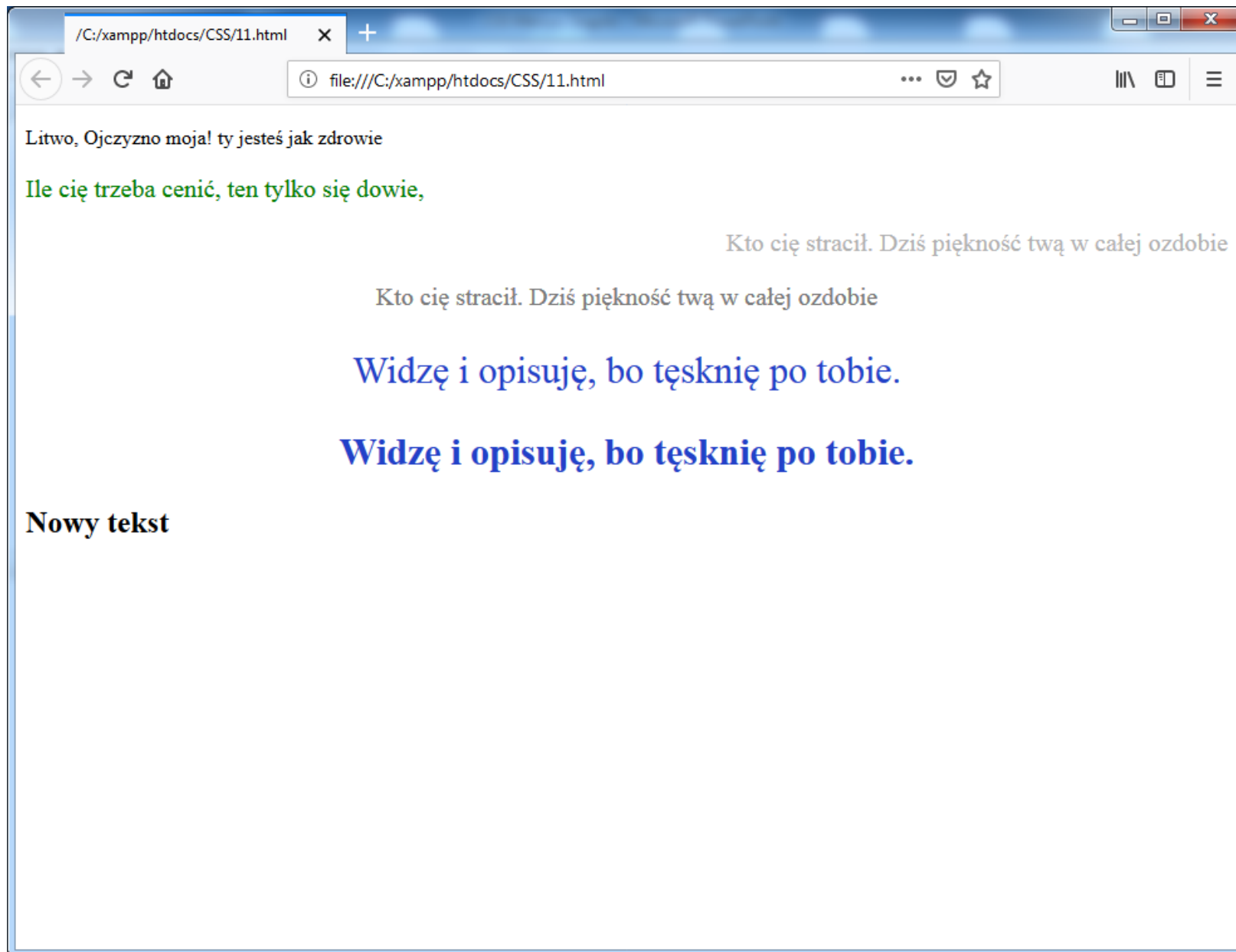
Widzę i opisuję, bo tęsknię po tobie.

Widzę i opisuję, bo tęsknię po tobie.

Selektor klasy

- Selektory pozwalają na nadanie określonych atrybutów takim samym elementom występującym na stronie. Ale czasami chcemy zastosować pewien styl formatowania do określonej grupy elementów, a do drugiej inny. Możemy wtedy dla każdej grupy utworzyć klasę o dowolnej nazwie i zdefiniować styl dla selektora klasy.
- Składnia selektora klasy:
`selektor.nazwa_klasy { właściwość: wartość }`
- Składnia uniwersalnego selektora klasy:
`.nazwa_klasy { właściwość: wartość }`

```
5 <style>
6   p.lewy{
7       font-size: 20px;
8       color: green;
9       text-align: left;
10    }
11   p.prawy{
12       font-size: 20px;
13       color: rgba(100,100,100,0.5);
14       text-align: right;
15    }
16   p.srodek{
17       font-size: 20px;
18       color: rgb(122,122,122);
19       text-align: center;
20    }
21   .inny{
22       font-size: 30px;
23       color: rgb(34,65,200);
24       text-align: center;
25    }
26 </style>
27 </head>
28 <body>
29 <p> Litwo, Ojczyzno moja! ty jesteś jak zdrowie</p>
30 <p class="lewy">Ile cię trzeba cenić, ten tylko się dowie,</p>
31 <p class="prawy">Kto cię stracił. Dziś piękność twą w całej ozdobie</p>
32 <p class="srodek">Kto cię stracił. Dziś piękność twą w całej ozdobie</p>
33 <p class="inny">Widzę i opisuję, bo tęsknię po tobie.</p>
34 <h2 class="inny">Widzę i opisuję, bo tęsknię po tobie.</h2>
35 <h2>Nowy tekst</h2>
```



Selektor klasy - zalety

- Możliwość tworzenia klas jest bardzo przydatna ponieważ możemy utworzyć klasę dotyczącą większej ilości selektorów, nie musimy widzieć jakiego selektora będzie dotyczyła definiowana klasa
- Łatwo można modyfikować wygląd strony
- Dla pojedynczego selektora możemy definiować kilka klas

Selektor identyfikatora

- Selektor identyfikatora jest stosowany, gdy chcemy nadać określone atrybuty formatowania elementowi, który ma przypisany jednoznaczny identyfikator, czyli występuje raz. Deklaracja identyfikatora musi zaczynać się znakiem #.
- Składnia:
selektor#identyfikator{właściwości: wartość;}
lub
#identyfikator{właściwości: wartość;}

```
1 <!DOCTYPE html> <!--12.html-->
2 <html lang="pl-PL">
3 <head>
4   <meta charset="UTF-8"/>
5   <style>
6     p#nowy{
7       font-size: 20px;
8       color: green;
9       text-align: left;
10    }
11  </style>
12 </head>
13 <body>
14   <p> Litwo, Ojczyzno moja! ty jesteś jak zdrowie</p>
15   <p id="nowy">Ile cię trzeba cenić, ten tylko się dowie,</p>
16   <p>Kto cię stracił. Dziś piękność twą w całej ozdobie</p>
17 </body>
18 </html>
19
```

/C:/xampp/htdocs/CSS/12.html



file:///C:/xampp/ht



Litwo, Ojczyzno moja! ty jesteś jak zdrowie

Ile cię trzeba cenić, ten tylko się dowie,

Kto cię stracił. Dziś piękność twą w całej ozdobie

Selektor pseudoklasy

W języku CSS można nadawać styl w związku z działaniem użytkownika. Przykładem takiego zastosowania może być odnośnik który zmienia swój wygląd po najechaniu na niego kursorem myszy, kliknięciu lub otwarciu linku. Element otrzymuje lub traci pseudoklasę podczas interakcji z użytkownikiem.

Właściwości odsyłaczy:

- :link** – link nieaktywny nie został odwiedzony przez użytkownika
- :visited** – link odwiedzony przez użytkownika
- :hover** – link gotowy do kliknięcia, kursor ustawiony nad linkiem (może być używany nie tylko dla odsyłaczy)
- :active** – link odwiedzony, strona jest obecnie wczytana

Selektor pseudoklasy

Składnia:

a:link{właściwość: wartość}

lub

a.nazwa_klasy:link{właściwość: wartość}

Selektor pseudoklasy 13.html

```
1  <!DOCTYPE html> <!--12.html-->
2  <html lang="pl-PL">
3  <head>
4  <meta charset="UTF-8"/>
5  <style>
6  a.nowy:link{
7      font-size: 10px;
8      color: green;
9  }
10 a.nowy:hover{
11     font-size: 20px;
12     color: yellow;
13 }
14 a.nowy:visited{
15     font-size: 30px;
16     color: rgb(22,32,24) ;
17 }
18 </style>
19 </head>
20 <body>
21
22 <a class="nowy" href="http:\\onet.pl"> Litwo, Ojczyzna moja! ty jesteś jak zdrowie</a>
23
24 </body>
25 </html>
26
```

Inne selektory pseudoklasy

:first-line – nadaje określone formatowanie
wszystkim pierwszym liniom

:first-letter – nadaje określone formatowanie
wszystkim pierwszym literom

Przykłady – selektory pseudoklasy

przykład

```
<!DOCTYPE html> <!--14.html-->
<html lang="pl-PL">
<head>
<meta charset="UTF-8"/>
<style>
div#zdjecie{
    background: url("kot.jpg") no-repeat;
    display: inline-block;
    width: 600px;
                                height: 600px;
}
div#zdjecie:hover {
    background: url("pies.jpg") no-repeat;
                                width: 600px;
                                height: 600px;

    /*rozjaśnianie*/
    opacity : 0.70;
    filter : alpha(opacity=70);
}
```

Przykłady – selektory pseudoklasy

```
a#prz{  
    background: url("p1.png") no-repeat;  
    display: inline-block;  
    width: 200px;  
        height: 200px;  
}  
a#prz:hover {  
    background: url("p2.png") no-repeat;  
        width: 200px;  
        height: 200px;  
}
```

Przykłady – selektory pseudoklasy

```
div#zdz{
    background: url("pies.jpg") no-repeat;
    display: inline-block;
    width: 600px;
        height: 600px;
}
div#zdz:hover {
    background: url("pies.jpg") no-repeat;
        width: 600px;
        height: 600px;

    /*rozjaśnianie*/
    opacity : 0.70;
    filter : alpha(opacity=70);
}

</style>
</head>
```

Przykłady – selektory pseudoklasy

```
<body>
```

```
<div id="zdjecie"></div>
```

```
<br>
```

```
<br>
```

```
<a id="prz" href="http:\\lukasiewicz.gorlice.pl"></a>
```

```
<div id="zdj"></div>
```

```
</body>
```

```
</html>
```

Formatowanie czcionki - CSS

font-family – służy do ustawiania rodzaju czcionki

Rodzaje czcionek:

serif – czcionki szeryfowe, mające wykończenia liter i cyfr np. Times New Roman, Georgia, Bodoni

sans-serif – czcionki bezszeryfowe. Końcówki są proste np. Arial, Futura, Verdana

monospace – czcionki monotypiczne. Znaki mają stałą szerokość np. Courier, Courier New

cursive – czcionki mają cechy czcionki pochylonej np. Comic Sans, Florence

fantasy – czcionki dekoracyjne np. Impact, OldTown

Formatowanie czcionki - CSS

```
p{  
font-family: Georgia, Courier, Arial;  
}
```

Po przecinku umieszczamy czcionki które mają być użyte w sytuacji gdy nie zostanie odnaleziona czcionka pierwsza na liście

Wielkość czcionki

font-size – określa wielkość czcionki

Przykład:

```
p{  
font-family: Courier;  
font-size: 12px  
}
```

Wielkość czcionki

Słowa kluczowe służące do określania wielkości czcionek:

xx-small

x-small

small

medium

large

x-large

xx-large

Wielkość czcionki

Można również zastosować słowa kluczowe
smaller i larger służą one do określenia
względnej wielkości czcionki

Styl czcionki

font-style: styl;

dostępne są trzy style:

normal – czcionka podstawowa

italic – czcionka pochylona

oblique – czcionka pochylona (działa na
wszystkie czcionki nawet na te co nie działa
italic)

Wariant czcionki

font-variant – pozwala wybrać między normalną czcionką a kapitalikami

dostępne opcje:

normal

small-caps

Waga czcionki

font-weight – waga czcionki

normal

bold

można też podać wartość np. 100, 200, 800, 1000

można używać również wartości względnych

lighter

bolder

Właściwość font

Można użyć właściwości font do określenia wszystkich parametrów czcionki lecz należy to wykonać w określonej kolejności:

```
selektor{ font: font-style font-variant font-weight font-size font-family;}
```

np.

```
p{font: italic bold 12px Georgia}
```

Niektóre atrybuty mogą zostać pominięte lecz nie można pominąć font-size i font-family

Formatowanie tekstu

- text-indent: wartość – powoduje ustawienie wcięcia pierwszego akapitu
 - np. `p{text-indent: 30px}`
 - text-align – służy do ustawienia wyrównania tekstu
 - dostępne opcje:
 - left
 - right
 - center
 - justify – rozłożenie tekstu równomiernie między marginesami
- np. `p{text-align: left;}`

Formatowanie tekstu

- line-height – odstęp między wierszami tekstu
możliwe opcje:
 - liczba – określa odstęp jako wielokrotność obecnego rozmiaru czcionki
 - wysokość – jest podawana z jednostką miary: px, pt, cm określa stałą wysokość odstępu
 - wartość procentowa – określa odstęp jako wysokość procentową obecnego rozmiaru czcionki

np. p {line-height: 10px;}

Formatowanie tekstu

- text-decoration – służy do dodawania efektów takich jak:

- none – tekst bez zmian
- underline – tekst podkreślony
- overline – linia umieszczona nad tekstem
- line-through – tekst przekreślony
- blink – migotanie tekstu

np. `p{text-decoration: underline;}`

można też łączyć

np. `p{text-decoration: underline line-through blink;}`

Formatowanie tekstu

- letter-spacing – służy do definiowania odstępu między literami
- word-spacing – odstęp między wyrazami

np. `p{letter-spacing: 3px;`

`word-spacing: 10px;}`

Formatowanie tekstu

- text-transform – służy do ustawiania wielkości liter tekst, dokonuje ich transformacji
 - capitalize – zamienia wszystkie pierwsze małe litery wyrazów na duże
 - uppercase – zamienia wszystkie litery na wielkie
 - lowercase – zamienia wszystkie litery na małe
 - none – bez transformacji

Formatowanie tekstu – białe znaki

- white-space – służy do sterowania wyświetlaniem na stronie spacji oraz innych białych znaków. Wstawianie w kodzie kilku spacji obok siebie spowoduje zamianę na pojedynczą spację. Również wystąpienie spacji na początku tekstu jest niemożliwe. Atrybut white-space pozwoli pozbyć się tych niedogodności.

Formatowanie tekstu – białe znaki

- normal – zamienia kilka sąsiednich spacji na jedną. Wiersze tekstu są przełamywane automatycznie.
- pre – umożliwia wyświetlanie kilku spacji obok siebie. Wiersze tekstu są przełamywane w momencie wystąpienia znaku nowej linii (Enter)
- nowrap – zamienia kilka sąsiednich spacji na jedną. Wiersze tekstu są przełamywane w miejscu wystąpienia znacznika `<br>`.

Formatowanie tekstu – białe znaki

- pre-wrap – umożliwia wyświetlenie kilku spacji obok siebie. Wiersze tekstu są przełamywane w miejscu wystąpienia znaku nowej linii (Enter) oraz automatycznie na końcu szerokości strony.
- pre-line – zamienia kilka spacji na jedną. Wiersze tekstu są przełamywane w miejscu wystąpienia znaku nowej linii (Enter) oraz automatycznie na końcu szerokości strony.

Czasem zdarza się że nie wszystkie przeglądarki prawidłowo interpretują ten atrybut.

Tło i kolor

- background-color – definiowanie koloru tła

```
body{  
background-color: red;  
}
```

- background-image: url("zdjecie.jpg") –
definiowanie obrazka tła

```
body{  
background-image: url("kot.jpg");  
}
```

Tło i kolor

Sposób przetwarzania grafiki w tle możemy określić atrybutem background-repeat.

repeat – powtarzanie tła w obu kierunkach

repeat-x – powtarzanie tła tylko w kierunku poziomym

repeat-y – powtarzanie tła tylko w kierunku pionowym

no-repeat – brak powtarzania tła

np. body{

background-image: url("kot.jpg");

background-repeat: no-repeat;

}

Tło i kolor

Aby zablokować grafikę w tle należy użyć atrybutu:
background-attachment

opcje:

scroll – przewijanie tła

fixed – tło jest nieruchome względem okna przeglądarki

local – tło nieruchome względem elementu dla którego zostało zdefiniowane

```
body{
```

```
background-attachment: fixed
```

```
}
```

Tło i kolor

Pozycję tła strony można określić przy pomocy argumentu `background-position`. Argument może przyjąć jedną lub dwie wartości.

jedna wartość:

`center` – środek

`left` – lewo

`right` – prawo

`top` – góra strony

`bottom` – dół strony

`30px` – 30 pikseli odstępu od lewego marginesu strony

Tło i kolor

Dwie wartości:

left top – lewy górny róg strony

left bottom – lewy dolny róg strony

center top – środek górny

itp.

Tło i kolor

Usunięcie tła może zostać wykonane przy pomocy atrybutu `background: none;`

Możliwe jest również podanie wszystkich wartości atrybutu `background` ale należy wykonać to w odpowiedniej kolejności

```
body{  
background: kolor tło_obrazkowe powtarzanie blokada_tła  
pozycja;  
}
```

Nie jest konieczne podawanie wszystkich wartości opuszczonym wartościom zostanie przypisana wartość domyślna. Nie wszystkie przeglądarki akceptują skrócony zapis.

Model blokowy CSS



Model blokowy CSS

Wszystkie elementy w języku HTML są traktowane jako prostokątne bloki, każdy z nich ma marginesy, obramowanie, wypełnienie i zawartość.

Margin – Jest to pusty obszar wokół ramki, nie ma koloru tła i jest przezroczysty.

Border – obramowanie wokół zawartości elementu, ma styl i kolor.

Padding – odstęp między obramowaniem i zawartością elementu

Content – zawartość elementu

Model blokowy CSS

Szerokość obramowania możemy ustawić przy pomocy atrybutu `border-width`.

Opcje:

Jedna wartość – szerokość dla wszystkich krawędzi

Dwie wartości – taka sama szerokość dla krawędzi poziomych i pionowych

Trzy wartości – pierwsza określa szerokość krawędzi górnej, druga krawędzi pionowych, trzecia krawędzi dolnej

Cztery wartości – określenie szerokości dla każdej z krawędzi osobno

`p{border-width: 12px 10px 11px 8px}` – określenie szerokości dla każdej krawędzi osobno

Model blokowy CSS

Szerokość krawędzi może być definiowana przy pomocy dowolnych jednostek lub przy pomocy następujących wartości:

- thin – cienkie obramowanie
- medium – średnie obramowanie
- thick – grube obramowanie

Model blokowy CSS

Można również definiować osobno szerokość poszczególnych krawędzi przy pomocy atrybutów:

- `border-top-width` – górna krawędź
- `border-bottom-width` – dolna krawędź
- `border-left-width` – lewa krawędź
- `border-right-width` – prawa krawędź

Model blokowy CSS

Styl obramowania definiujemy za pomocą atrybutu `border-style` i możemy zrobić to podobnie jak w przypadku szerokości dla wszystkich krawędzi lub podając większą ilość dla poszczególnych krawędzi.

- `border-style`
- `border-top-style`
- `border-bottom-style`
- `border-left-style`
- `border-right-style`

Model blokowy CSS

Właściwości które możemy przypisać do stylów obramowania:

- none – brak obramowania
- hidden – obramowanie ukryte
- dotted – linia kropkowana
- dashed – linia kreskowana
- solid – linia ciągła
- double – linia ciągła podwójna
- groove – rowek
- ridge – grzbiet
- inset – ramka
- outset – przycisk

Model blokowy CSS

Kolor obramowania ustawiamy przy pomocy atrybutu `border-color`.

Oczywiście tak jak w poprzednich przypadkach możliwe jest podanie kilku wartości obok siebie które będą odpowiadały odpowiednim krawędziom. Można też użyć osobnych atrybutów dla każdej krawędzi:

- `border-top-color`
- `border-bottom-color`
- `border-left-color`
- `border-right-color`

Model blokowy CSS

Do definiowania obramowania można użyć jednego atrybutu border i zdefiniować poszczególne wartości. Kolejność definiowania wartości jest dowolna.

```
np. p{border: double 10px blue;}
```

```
h1{border: 30pt outset red;}
```

```
table{border: thick dotted black;}
```

Model blokowy CSS – marginesy zewnętrzne

Margines zewnętrzny określa przestrzeń wokół definiowanego elementu. Przestrzeń ta oddziela element od innych elementów strony.

Dostępne atrybuty:

margin-top

margin-bottom

margin-left

margin-right

Model blokowy CSS – marginesy zewnętrzne

Można również użyć atrybutu margin.

- jedna wartość – wszystkie marginesy jednakowe
- dwie wartości – pierwsza wartość dolny i górny, druga prawy i lewy
- trzy wartości – pierwsza górny, druga lewy prawy, trzecia dolny
- cztery wartości – każdy margines osobno

np. `p{margin: 1cm 2cm 3cm 4cm;}`

Model blokowy CSS – marginesy zewnętrzne

Przykłady użycia:

```
p{margin-left: 12px;}
```

p{margin-top: auto;} – przeglądarka dostosuje automatycznie margines

```
h1{margin-bottom: 0;}
```

Model blokowy CSS – marginesy wewnętrzne

Margines wewnętrzny (padding) określa przestrzeń między zawartością elementu a obramowaniem.

Dostępne atrybuty:

padding-top

padding-bottom

padding-left

padding-right

Model blokowy CSS – marginesy wewnętrzne

Można również użyć atrybutu padding.

- jedna wartość – wszystkie marginesy jednakowe
- dwie wartości – pierwsza wartość dolny i górny, druga prawy i lewy
- trzy wartości – pierwsza górny, druga lewy prawy, trzecia dolny
- cztery wartości – każdy margines osobno

np. `p{padding: 10px 11px 12px 12px;}`

Model blokowy CSS – marginesy wewnętrzne

Przykłady użycia:

```
p{padding-left: 12px;}
```

p{padding-top: auto;} – przeglądarka dostosuje automatycznie margines

```
h1{padding-bottom: 0;}
```

Model blokowy CSS – obrys

Obramowanie wokół elementu można tworzyć przy pomocy obrysu. Obrys różni się od obramowania tym:

- nie zajmuje miejsca, dzięki czemu nie wpływa na rozmiar i położenie elementu
- jest tworzony na wierzchu elementu
- jest definiowany dla wszystkich krawędzi równocześnie

Model blokowy CSS – obrys

Obrys tworzy się przy pomocy atrybutu outline.

Dostępne atrybuty:

- outline
- outline-style
- outline-width
- outline-color

Model blokowy CSS – obrys

np.

```
p{outline: solid 3px blue;}
```

```
p{outline-style: solid}
```

Jeżeli używamy atrybutu outline to obowiązkowym jest pierwszy parametr, pozostałe dwa możemy pominąć.

Atrybuty związane z rozmiarem elementów

width – szerokość

min-width – określa minimalną szerokość elementu. Pozwala ustawić szerokość tak aby nigdy nie była mniejsza od podanej wartości

max-width – określa maksymalną szerokość elementu. Element może być węższy ale nigdy nie może być szerszy

np.

```
img{max-width: 300px;}
```

Atrybuty związane z rozmiarem elementów

height – wysokość

min-height – określa minimalną wysokość elementu. Pozwala ustawić wysokość tak aby nigdy nie była mniejsza od podanej wartości

max-height – określa maksymalną wysokość elementu. Element może być wyższy ale nigdy nie może być niższy

np.

```
img{max-height: 300px;}
```

Atrybuty związane z rozmiarem elementów

overflow – atrybut przepełnienia

opcje:

- visible – pokazywana jest cała zawartość niezależnie od ustawionego rozmiaru
- hidden – treść nie mieszcząca się w danym rozmiarze zostanie ukryta
- scroll – powoduje w razie potrzeby wyświetlenie się paska przewijania
- auto – jeżeli jest to konieczne suwaki zostaną wyświetlone

Atrybuty związane z rozmiarem elementów

Atrybutu `overflow` można używać tylko dla elementów blokowych ze zdefiniowaną wysokością. Domyślną wartością dla argumentu `overflow` jest `visible`.

Formatowanie list - CSS

Selektory możliwe do formatowania w listach to `<ul>`, `<ol>`, `<li>`. Atrybut `list-style-type`.

Możliwe wartości atrybutów:

- `disc` – koło
- `circle` – okrąg
- `square` – kwadrat
- `decimal` – liczby arabskie
- `lower-roman` – małe cyfry rzymskie
- `upper-roman` – duże cyfry rzymskie
- `lower-alpha` – małe litery
- `upper-alpha` – duże litery
- `none` – brak punktora

Formatowanie list - zdjęcie

Aby zamiast punktu wyświetlić zdjęcie w liście wypunktowanej należy użyć znacznika `ul` oraz atrybutu `list-style-image: url(nazwa.png)`

Aby ustawić położenie punktora należy użyć atrybutu `list-style-position`.

Możliwe wartości:

- outside
- inside

Tabele – formatowanie w CSS

caption-side – służy do określenia położenia podpisu dodanego do tabeli

możliwe wartości :

- top
- bottom
- left
- right

np.

```
table{caption-side: bottom;}
```

Tabele – formatowanie w CSS

`border` – służy do określenia obramowania, w przypadku tabeli można go stosować do selektorów `table`, `th`, `td`, `tr`

np. `table{border: 5px solid black;}` – cała tabela będzie obramowana w następujący sposób

lub

`td{border: 5px solid black;}` – tylko wiersze będą obramowane w następujący sposób

Tabele – formatowanie w CSS

`border-collapse` – określa sposób obramowania komórek tabeli. Możliwe jest obramowanie sąsiednich komórek za pomocą jednej linii lub obramowanie każdej komórki osobno.

możliwe wartości to:

`collapse` – pojedyncze obramowanie

`separate` – komórki tabeli będą od siebie odseparowane

```
np. table{  
border-collapse: collapse;  
}
```

Tabele – formatowanie w CSS

empty-cells – służy do określenia obramowania pustych cel

możliwe wartości:

- show
- hide

Lecz należy pamiętać żeby ta opcja działała należy ustawić border-collapse na separate

Tabele – formatowanie w CSS

table-layout – służy do rozplanowania tabeli na stronie

możliwe wartości:

auto – szerokość tabeli jest ustawiona automatycznie i zależy od zawartości komórek

fixed – szerokość tabeli wynika z zadeklarowanej szerokości poszczególnych komórek oraz szerokości całej tabeli

Uwaga !!! różne przeglądarki różnie interpretują ten atrybut

Tabele – formatowanie w CSS

`border-spacing` – służy do określania wielkości odstępu między komórkami. Gdy podamy jedną wartość będzie dotyczyło to wszystkich krawędzi, gdy podamy dwie wartości pierwsza będzie dotyczyła odstępu poziomego, druga będzie dotyczyła odstępu pionowego.

np.

```
#table1 {  
    border-collapse: separate;  
    border-spacing: 15px;  
}  
  
#table2 {  
    border-collapse: separate;  
    border-spacing: 15px 50px;  
}
```

Tabele – formatowanie w CSS

width – szerokość

height – wysokość

Jeżeli podane atrybuty zastosujemy w selektorach table, th, td, tr ustawimy sobie odpowiednią wysokość i szerokość tabeli

Pozycjonowanie w CSS

Pozycjonowanie na stronach internetowych pozwala na zdefiniowanie położenia elementu względem okna przeglądarki lub innego elementu.

Do pozycjonowania służy atrybut position

Składnia:

```
selektor{position: rodzaj; parametry;}
```

Pozycjonowanie w CSS

Atrybut position może przyjmować następujące wartości:

- static
- relative
- absolute
- fixed

Pozycjonowanie w CSS

Static przywraca normalne pozycjonowanie elementu. Jest to wartość przydatna wtedy, gdy wcześniej została podana deklaracja pozycjonowania tego typu elementów. Użycie w definicji stylu inline dla wybranego elementu deklaracji `position: static` zniesie wcześniej zadeklarowane wartości w kodzie CSS dla danego elementu.

Pozycjonowanie w CSS

Relative – jest to pozycjonowanie względne. Pozwala przesunąć wybrany element w inne miejsce w stosunku do położenia pierwotnego. Poprzez ustawienie parametrów można określić sposób przesunięcia

Pozycjonowanie w CSS

Absolute – pozwala przesunąć element w inne miejsce względem brzegów strony lub bloku. Poprzez ustawienie parametrów można określić sposób przesunięcia. Element zostanie umieszczony na sztywno w miejscu określonym przez parametry.

Pozycjonowanie w CSS

fixed – działa podobnie jak absolute. Różnica polega tylko na tym że pozycjonowanie fixed ustala położenie elementu zawsze względem okna przeglądarki. Tak pozycjonowany element pozostaje nieruchomy przez cały czas, nawet jeżeli będziemy przewijali okno przeglądarki.

Pozycjonowanie w CSS

Parametry jakie można podawać podczas pozycjonowania:

- ✓ left – przesunięcie o określoną wartość w stosunku do lewej krawędzi położenia pierwotnego
- ✓ right - przesunięcie o określoną wartość w stosunku do prawej krawędzi położenia pierwotnego
- ✓ top - przesunięcie o określoną wartość w stosunku do górnej krawędzi położenia pierwotnego
- ✓ bottom - przesunięcie o określoną wartość w stosunku do dolnej krawędzi położenia pierwotnego

Pozycjonowanie w CSS

```
np. img{    position: absolute;
            left: 100px;
            top: 50px;
            }
```

Wartości podajemy w pixelach, można też wpisać wartość auto – wtedy zostanie ustawiona wartość domyślna.

Należy pamiętać że atrybuty left i top mają pierwszeństwo w stosunku do right i bottom.

Przezroczystość w CSS

Do ustawiania przezroczystości służy atrybut `opacity`. Może przyjmować wartości od 0 do 1. Zero oznacza całkowitą przezroczystość a jeden brak przezroczystości. Np. 0.1 to jest obiekt widoczny z przezroczystością 90%.

np. `img {opacity: 0.5;}` – oznacza że obiekt będzie widoczny z przezroczystością 50%

Pozycjonowanie w CSS – 24.html

Wyrównanie w pionie można ustawić przy pomocy atrybutu vertical-align.

Wyrównanie odbywa się w stosunku do innych elementów strony.

Wartości atrybutu vertical-align:

- baseline – wyrównuje linię bazową elementu do linii bazowej elementu nadrzędnego
- middle – ustawienie elementu na środku wysokości sąsiednich elementów
- text-top – wyrównuje górną krawędź elementu do górnej krawędzi tekstu elementu nadrzędnego
- text-bottom – wyrównuje podstawę elementu do podstawy tekstu elementu nadrzędnego
- super – ustawia element jako indeks górny
- sub – ustawia element jako indeks dolny
- top – wyrównuje górną krawędź elementu do górnej krawędzi najwyższego elementu linii
- bottom – wyrównuje podstawę elementu do podstawy elementu położonego najniżej w linii

Pozycjonowanie w CSS

np. `p{vertical-align: middle;}`

oprócz wyżej wymienionych właściwości można również podać wyrównanie w np. w mm.

`p{vertical-align: 2mm;}`

Pozycjonowanie w CSS

Do ustawienia elementów w poziomie służy atrybut float.

Float może przyjmować następujące wartości:

- left – element może zostać ustawiony po lewej stronie względem elementów sąsiadujących
- right - element może zostać ustawiony po prawej stronie względem elementów sąsiadujących
- none – element nie będzie sąsiadował z innymi elementami

Pozycjonowanie w CSS

Atrybut clear – ustawia pozycję kolejnego elementu względem elementu pływającego.

Wartość elementu clear określa sposób zachowania się kolejnych elementów względem elementu pływającego.

Możliwe wartości:

- left – lewy bok następnego elementu nie przylega do elementu pływającego
- right – prawy bok następnego elementu nie przylega do elementu pływającego
- both – żaden bok następnego elementu nie przylega do elementu pływającego
- none – brak ograniczeń

Sposoby wyświetlania elementów w CSS

Wszystkie zdefiniowane elementy są wyświetlane na stronie w sposób domyślny. Jeżeli chcemy zmienić ten sposób musimy użyć atrybutu display.

Parametr display przyjmuje następujące wartości:

- block – element jest wyświetlany jako element blokowy z odstępami od góry i od dołu
- inline – element wyświetlany jest w linii z innymi dokumentami
- list-item – element wyświetlany jest jako element wykazu listy
- none – element nie będzie wyświetlany
- inline-block – element wyświetlany podobnie jak element zastępowany, mimo że element ma charakter blokowy jest wyświetlany jako liniowy
- run-in – jeżeli po elemencie występuje element będący blokiem, to ten pierwszy element zostanie wyświetlony w jednej linii z blokiem, w przeciwnym razie element zostanie wyświetlony w bloku.

Sposoby wyświetlania elementów w CSS

Elementami blokowymi są:

- `<div>`
- `<h1> ... <h6>`
- `<p>`
- `<form>`
- `<header>`
- `<footer>`
- `<section>`

Zajmują całą szerokość strony i zawsze zaczynają się od nowej linii.

Sposoby wyświetlania elementów w CSS

Elementy wbudowane inline:

- `<span>`
- `<img>`
- `<a>`

Menu rozwijane

```
<!DOCTYPE html>
<html lang="pl-PL">
<head>
<meta charset="UTF-8"/>
<style>
body{
background-color: black;
}

ul, ul li { margin: 0;
padding: 0;}

#menu1 li{
list-style: none;
}

#menu1, #menu1 ul {
width: 140px;
}
```

Menu rozwijane

```
#menu1 ul {position: absolute;
            top: 0;
            left: 160px;
            padding-left: 1px;
            visibility: hidden;}

#menu1 li {padding-bottom: 1px;
            position: relative;}

#menu1 li:hover ul {background-color: #ff0000;
                    visibility: visible;
                    width: auto;
                    height: auto;
                    }

#menu1 li:hover ul li:hover a {background-color: #0000ff;
                                }
```

Menu rozwijane

```
#menu1 a, #menu1 li:hover li a {display: block;
font: 12px/30px arial, verdana;
text-decoration: none;
padding: 0px 10px;
width: 140px;
color: red;
background-color: #ffffff;

}

#menu1 li:hover a, #menu1 li:hover li:hover li:hover a {
background-color: #ffff00;
color: white;
}

</style>
```

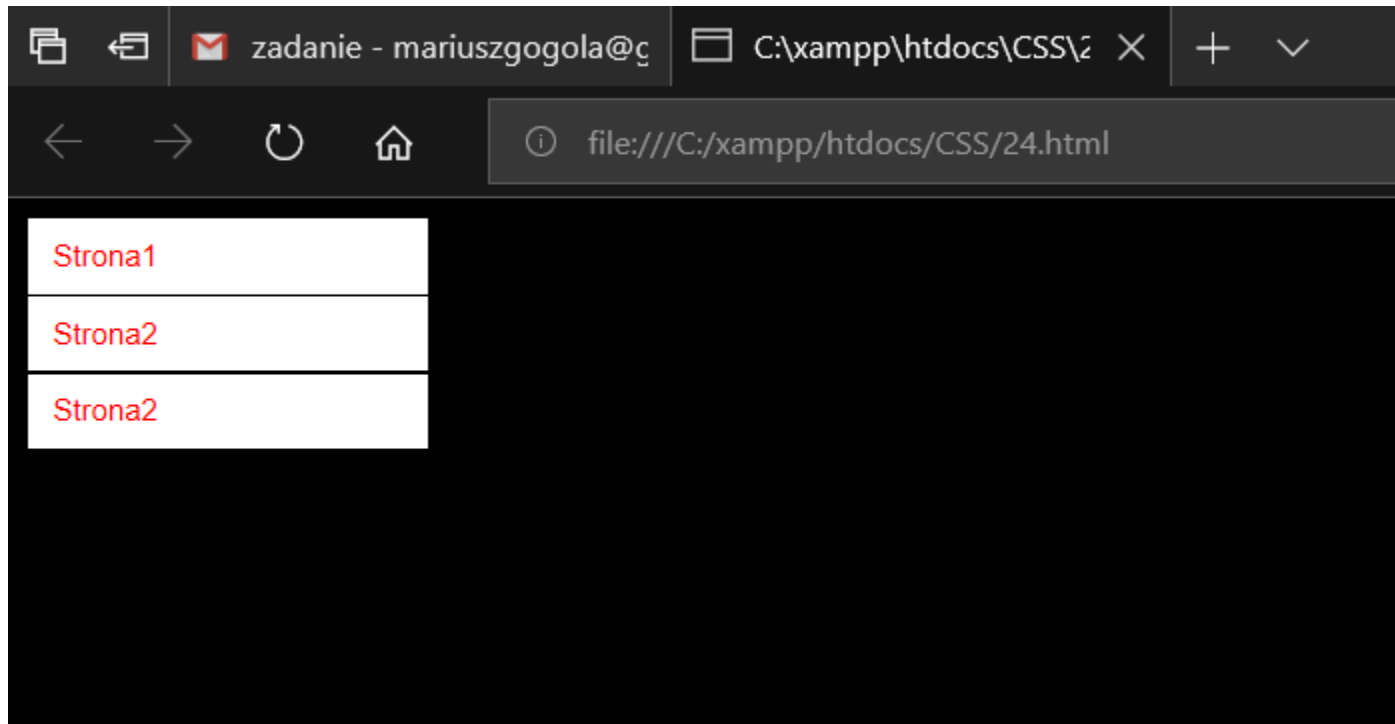
Menu rozwijane

```
</head>
<body>
<ul id="menu1">
<li><a href="s1.html">Strona1</a></li>
<li><a href="s1.html">Strona2</a>
<ul>
<li><a href="s1.html">Podstrona1</a></li>
<li><a href="s1.html">Podstrona2</a></li>
<li><a href="s1.html">Podstrona3</a></li>
<li><a href="s1.html">Podstrona4</a></li>
</ul>
</li>
<li><a href="s1.html">Strona2</a></li>

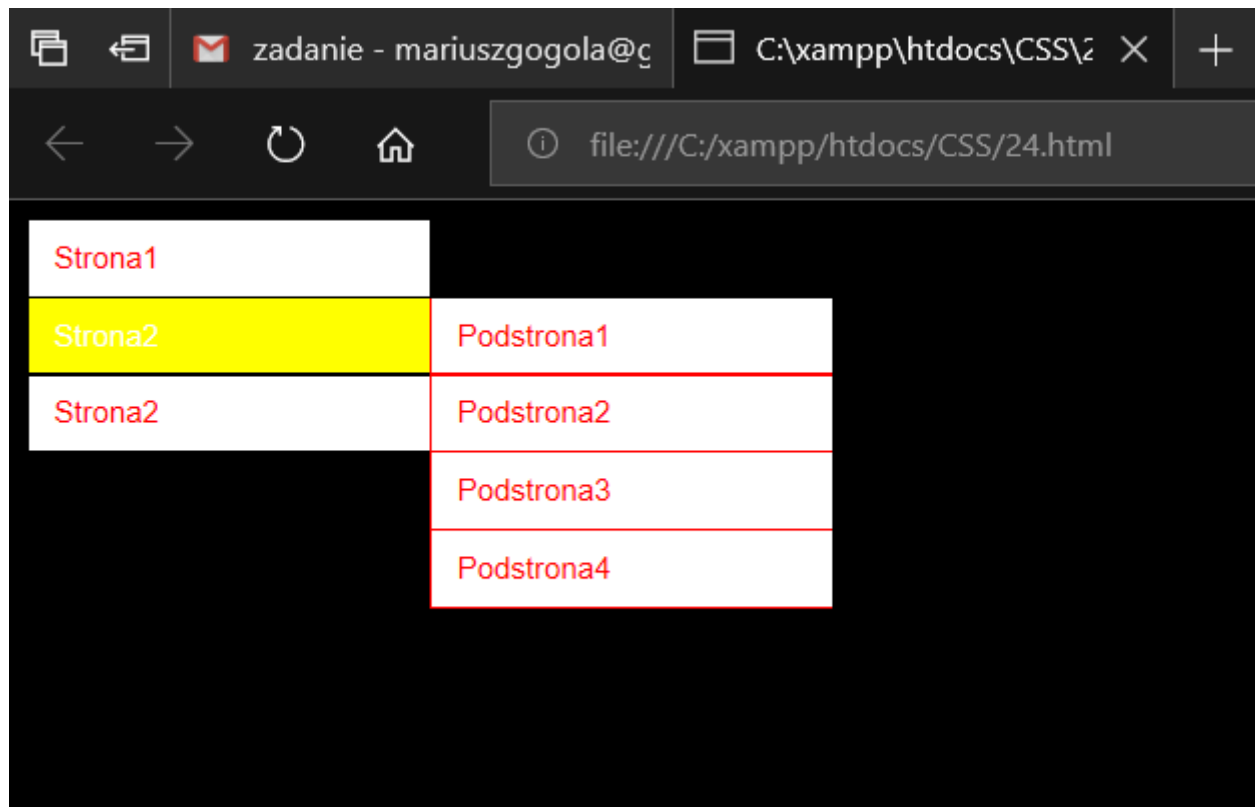
</ul>

</body>
</html>
```

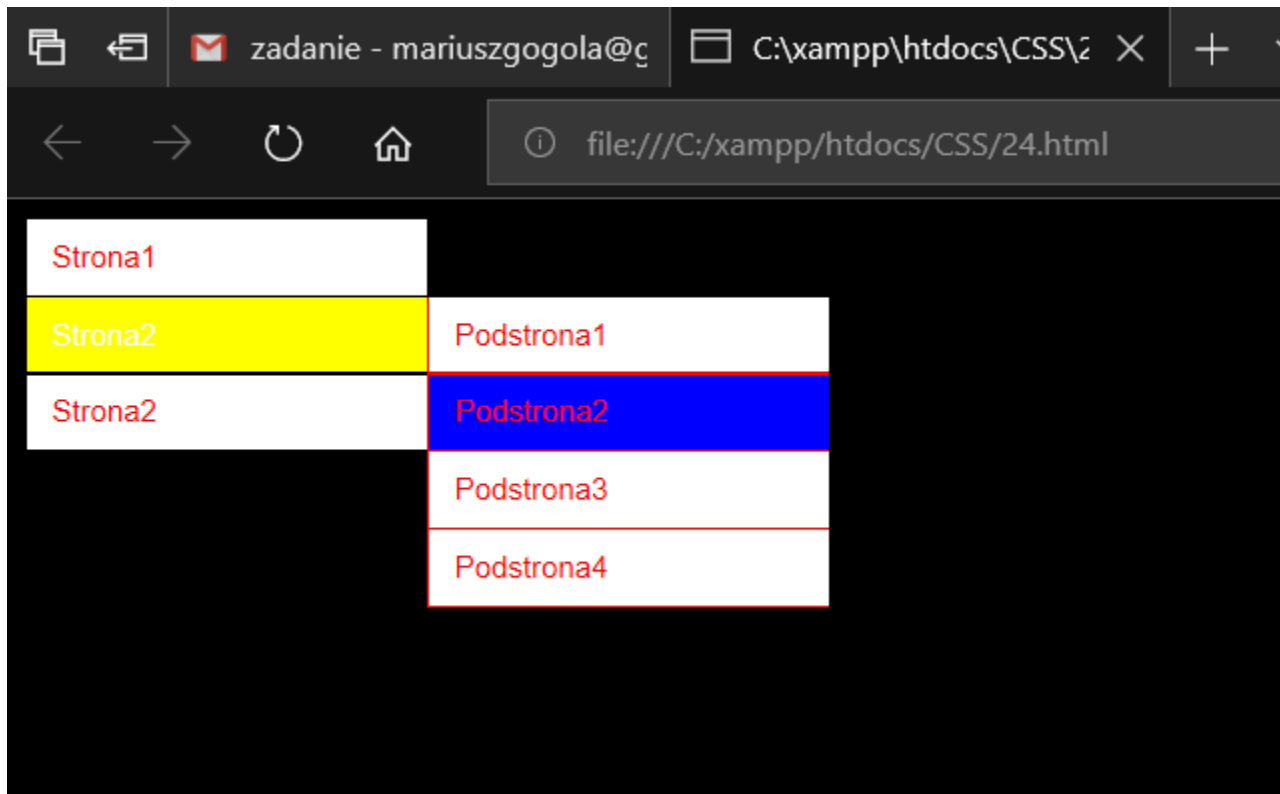
Menu rozwijane



Menu rozwijane



Menu rozwijane



Szablon strony

Obecnie standardem projektowania szablonu stron jest szerokość ekranu 1024, czyli szerokość strony pasuje ustawić mniejszą np. 1000, oczywiście większość monitorów posiada dużo większą rozdzielczość ale musimy dostosować się do użytkowników posiadających starszy sprzęt tak aby i oni mogli wygonie przeglądać naszą stronę.

Szablon strony

Na stronie należy używać znaczników ułatwiających rozmieszczenie elementów:

`<div>` `<footer>` `<nav>` `<header>` `<section>`

W praktyce najczęściej używanym szablonem strony jest nagłówek, trzy kolumny, stopka.

Równie często spotyka się układ nagłówek, dwie kolumny i stopka.

Szablon strony

```
<!DOCTYPE html>
<html lang="pl-PL">
<head>
<meta charset="UTF-8"/>
<style>

    </style>
</head>
<body>
<header id="naglowek"> Nagłówek
    <nav>
        menu nawigacja po stronie
    </nav>
</header>
```

Szablon strony

```
<div id="ramka">  
  <section class="kolumny" id="kol1">  
    <article class="art" id="k1a1">  
      A1 w K1  
    </article>  
    <article class="art" id="k1a2">  
      A2 w K1  
    </article>  
    <article class="art" id="k1a3">  
      A3 w K1  
    </article>  
  </section>
```

Szablon strony

```
<section class="kolumny" id="kol2">  
  <article class="art" id="k2a1">  
    A1 w K2  
  </article>  
  <article class="art" id="k2a2">  
    A2 w K2  
  </article>  
  <article class="art" id="k2a3">  
    A3 w K2  
  </article>  
</section>
```

Szablon strony

```
<section class="kolumny" id="kol3">  
  <article class="art" id="k3a1">  
    A1 w K3  
  </article>  
  <article class="art" id="k3a2">  
    A2 w K3  
  </article>  
  <article class="art" id="k3a3">  
    A3 w K3  
  </article>  
</section>
```

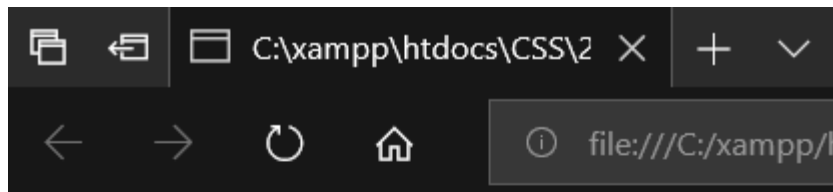
Szablon strony

```
<footer id="stopka"> TO JEST STOPKA  
</footer>
```

```
</div>
```

```
</body>  
</html>
```

Szablon strony – efekt działania – bez CSS



Nagłówek
menu nawigacja po stronie
A1 w K1
A2 w K1
A3 w K1
A1 w K2
A2 w K2
A3 w K2
A1 w K3
A2 w K3
A3 w K3
TO JEST STOPKA

Szablon strony

Dołączamy nasz styl:

```
<link rel="stylesheet" type="text/css"  
href="css25.css">
```

Szablon strony – plik CSS

```
*{  
    margin: 0px;  
    padding: 0px;  
}  
#ramka {  
    width: 1000px;  
    margin-right: auto;  
    margin-left: auto;  
}  
  
.kolumny {  
    float: left; /*Każda z kolumn będzie dołączana  
                                                         z lewej strony*/  
    width: 30%;  
    margin-top: 10px;  
    margin-left: 10px;  
    margin-right: 2%;  
    margin-bottom: 2%;  
}
```

Szablon strony – plik CSS

```
#stopka{
    clear: both;      /*Podowduje że stopka wyświetli
                        się od lewej krawędzi strony
                        a nie następna z lewej po
                        kolumnach*/

    width: 100%;
    background-color: #FCF;
}
section {
    overflow: hidden; /*ukrywa to co nie mieści się w kolumnie
                        zachodzeniu jednej kolumny
                        na drugą*/
}
```

Szablon strony – plik CSS

```
#naglowek {  
    width: 100%;  
    background-color: #FCF;  
}
```

```
#kol1{  
    float: left;  
    background-color: #FCF;  
}
```

```
#kol2{  
    float: left;  
    background-color: #FCF;  
}
```

```
#kol3{  
    float: left;  
    background-color: #FCF;  
}
```

Szablon strony



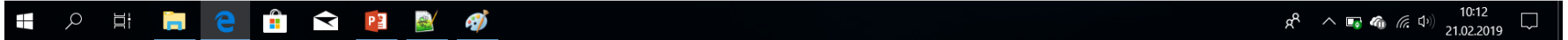
Nagłówek
menu nawigacja po stronie

A1 w K1
A2 w K1
A3 w K1

A1 w K2
A2 w K2
A3 w K2

A1 w K3
A2 w K3
A3 w K3

TO JEST STOPKA



Responsywny układ strony

Responsywny układ strony to taki który zmienia swoje atrybuty za pomocą stylów CSS w zależności od różnych zewnętrznych czynników. Najczęstszym czynnikiem wpływającym na wygląd strony jest szerokość okna przeglądarki. Dzięki zdefiniowaniu osobnych reguł dla różnych wielkości okna przeglądarki.

Responsywny układ strony pozwala za pomocą kodu HTML i CSS zmienić rozmiar zawartości strony, przesunąć określoną zawartość lub ją ukryć, tak aby strona wyglądała dobrze na dowolnym urządzeniu.

Responsywny układ strony

W HTML za pomocą znacznika <meta> możliwe jest dołączenie metatagu viewport i przejęcie kontroli nad widocznym dla użytkownika obszarem wyświetlania strony internetowej.

```
<meta name= "viewport" content="width=device-width, initial-scale=1.0">
```

Responsywny układ strony

`width=device-width` – określa szerokość strony która powinna odpowiadać szerokości ekranu urządzenia, gdzie będzie wyświetlana strona

`initial-scale=1.0` – ustawia początkowy poziom powiększenia gdy strona otwierana jest przez przeglądarkę.

Responsywny układ strony

W specyfikacji CSS3 rozbudowane zostały queries media. Dodając regułę @media, spowodujemy włączenie określonych atrybutów stylów CSS tylko wtedy gdy spełnione zostaną określone warunki.

Przykład: <C:\xampp\htdocs\CSS\29.html>

Responsywny układ strony

```
<!DOCTYPE html> <!--12.html-->
<html lang="pl-PL">
<head>
<meta charset="UTF-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<style>
body{
    background-color: red;
}

@media only screen and (max-width: 500px){
    body{
        background-color: blue;
    }
}
</style>
</head>
<body>

<p>NASTĄPI ZMIANA KOLORU TŁA </p>

</div>

</body>
</html>
```

Responsywny układ strony

Efektem działania powyższego kodu jest zmiana koloru tła strony gdy rozdzielczość okna przeglądarki zmniejszy się poniżej 500px.

Strony wyświetlane na telefonach komórkowych powinny mieć inny układ niż strony wyświetlane na ekranach monitorów.

Responsywny układ strony

Responsywny układ strony